

Project Story — Project Overheat: Predictive Maintenance AI

The Beginning

The project started with a simple but important industrial problem: machines usually tell us something is wrong before they actually fail, but that information is hidden inside continuous streams of sensor data.

The scenario was based on AeroLogistics, where automated warehouse sorting robots depend on high-speed drive motors. Unexpected motor failures can interrupt operations and create significant downtime costs.

The challenge was therefore not simply to classify machines as "healthy" or "failed."

The real question was:

Can sensor data provide enough warning to predict a catastrophic motor failure 24 hours before it happens?

The Data

The project used 10,000 hourly IoT telemetry records collected between January 2023 and February 2024.

Each observation contained:

- Temperature
- Vibration
- Voltage
- Catastrophic failure status

The biggest challenge appeared immediately: only 15 of the 10,000 observations represented catastrophic failures.

That meant the failure class represented just 0.15% of the dataset.

A model could achieve impressive-looking accuracy simply by predicting "no failure" almost every time.

So the project required a different approach.

Turning Raw Sensor Data Into Machine Health Signals

A single sensor spike does not necessarily mean a machine is about to fail.

Mechanical degradation is often a pattern that develops over time.

To capture that behavior, I created two time-series features:

- **A 12-hour rolling mean of temperature**
- **A 12-hour rolling standard deviation of vibration**

The idea was to capture both the machine's sustained thermal behavior and changes in mechanical vibration.

This was one of the most important parts of the project because the raw sensor readings alone do not necessarily describe machine health over time.

The 24-Hour Prediction Challenge

The next challenge was defining what "predict failure in advance" actually means.

If the model learned directly from the catastrophic-failure column, it would essentially learn to recognize failure at the time it occurs.

That would not give the maintenance team enough time to act.

Instead, the target was shifted by 24 hours.

The model was therefore trained to answer:

"Will this machine experience a catastrophic failure 24 hours from now?"

This transformed the project from failure detection into failure prediction.

Avoiding the Time-Series Leakage Trap

Another major lesson was that normal random train/test splitting is inappropriate for this type of problem.

With time-series data, randomly mixing observations can allow the model to learn from future behavior while predicting earlier behavior.

To avoid this, the dataset was split chronologically:

First 80% → Training

Final 20% → Testing

This made the evaluation much closer to how the model would actually be used in the real world.

Building the Model

A class-balanced Random Forest Classifier was selected for the predictive model.

Because failures were extremely rare, class balancing was used so that the model would not simply ignore the minority failure class.

The final test set contained 1,993 observations, including only three actual failure events.

The model achieved:

- 99.75% accuracy
- 66.67% failure recall
- 33.33% failure precision
- 44.44% F1 score
- 99.63% ROC-AUC

The confusion matrix showed:

- 1,986 correctly identified non-failures
- 2 correctly identified failures
- 1 missed failure
- 4 false alarms

The Most Interesting Discovery

The model's feature importance provided an interpretable answer to an important maintenance question:

Which sensor is actually giving us the strongest warning?

The answer was vibration.

Feature importance ranked:

1. Sensor Vibration — 38.90%
2. 12-Hour Temperature Rolling Mean — 26.55%
3. Sensor Temperature — 25.98%
4. 12-Hour Vibration Rolling Standard Deviation — 5.51%
5. Sensor Voltage — 3.06%

This meant vibration was the strongest predictive signal in this particular dataset, while temperature and its longer-term trend also played significant roles.

From Model to Business Decision

The project did not stop at model metrics.

The final objective was to turn the prediction into an operational action.

If a machine receives a high-risk prediction, the maintenance team should investigate it before the predicted 24-hour failure window.

That could allow maintenance to be planned rather than waiting for a catastrophic failure.

The project therefore represents a transition from:

Reactive Maintenance

to

Predictive Maintenance

What This Project Taught Me

The biggest lesson was that successful Data Science is not just about choosing an algorithm.

The important work happened around the model:

Understanding the business problem → engineering meaningful time-series features → defining the correct future target → preventing leakage → handling rare events → evaluating the right metrics → explaining the model → converting predictions into action.

The project also taught me why a 99%+ accuracy score can be misleading when failures are rare.

For predictive maintenance, missing a real failure can be much more important than correctly identifying thousands of normal operating hours.

The Story I Want to Carry Forward

Project Overheat represents my approach to Data Science:

Start with a real operational problem.

Understand the data-generating process.

Engineer features that represent the underlying behavior.

Build the model carefully.

Validate it honestly.

Explain the result.

And finally, turn the prediction into a decision.

That is the difference between building a machine-learning model and building a useful Data Science solution.